Requisitos funcionales

1. Gestión de usuarios

Permitir el registro y autenticación de usuarios mediante credenciales:

- Diseñar la interfaz de registro e inicio de sesión.
- Crear la clase de usuario.
- Implementar el registro de nuevos usuarios.
- Implementar la validación de credenciales.
- Implementar el almacenamiento y recuperación de los datos del usuario.
- Mostrar mensajes cuando las credenciales sean incorrectas.
- Permitir diferenciar entre usuarios normales y administradores mediante un rol.

Permitir al usuario seleccionar su perfil: estudiante, adulto mayor, personal administrativo u otro:

- Definir los tipos de perfiles disponibles.
- Diseñar la interfaz de selección de perfil.
- Asociar el perfil seleccionado con el usuario.
- Almacenar el perfil seleccionado.
- Realizar pruebas de selección y almacenamiento del perfil.

Permitir al usuario consultar y actualizar su información:

- Mostrar la información del usuario autenticado.
- Permitir modificar la información disponible del perfil.
- Guardar los cambios realizados.
- Mostrar un mensaje indicando si los cambios fueron realizados correctamente.

Permitir al usuario actualizar su contraseña:

- Solicitar la información necesaria para realizar el cambio.
- Validar los datos ingresados.
- Actualizar la contraseña del usuario.
- Mostrar un mensaje indicando si el cambio fue realizado correctamente.

2. Inicio y navegación

Permitir iniciar una partida nueva o continuar una partida guardada:

- Diseñar el menú principal.
- Implementar la opción de iniciar una nueva partida.
- Inicializar el puntaje, intentos y progreso de una nueva partida.
- Implementar la opción de continuar una partida.
- Recuperar los datos de una partida guardada.

Mostrar las instrucciones y reglas del juego:

- Redactar las instrucciones y reglas.
- Diseñar la pantalla de instrucciones.
- Implementar la navegación hacia la sección de instrucciones.
- Implementar la opción de regresar al menú principal.

Permitir seleccionar entre los niveles que el jugador tenga desbloqueados:

- Diseñar la pantalla de selección de niveles.
- Identificar los niveles desbloqueados por el jugador.
- Mostrar los niveles disponibles y bloqueados.
- Impedir el acceso a niveles que todavía no estén desbloqueados.
- Permitir iniciar un nivel disponible.
- Consultar la información de los niveles disponibles.
- Consultar la información específica del nivel seleccionado.

3. Desarrollo del juego

Presentar escenarios simulados de posibles estafas mediante correos, mensajes, redes sociales, sitios web, enlaces o archivos:

- Definir la estructura de los escenarios.
- Crear los diferentes tipos de escenarios con características que diferencien uno de escenario de otro.
- Almacenar el contenido y características de cada escenario.
- Clasificar los escenarios según su tipo y nivel.
- Implementar la selección de escenarios.
- Diseñar la interfaz donde se mostrará cada situación.
- Asociar cada escenario con el nivel al que pertenece.
- Definir el orden en el que se presentan los escenarios dentro de un nivel. ● Permitir que los escenarios incluyan información como remitente, destinatario, asunto, mensaje, enlace, archivo o imagen según corresponda.
- Conectar la vista de los escenarios con las funciones correspondientes del backend.

Permitir que el jugador clasifique cada escenario como seguro o fraudulento:

- Diseñar las opciones de respuesta.
- Registrar la selección realizada por el jugador.
- Enviar la respuesta seleccionada para su evaluación.
- Evitar respuestas múltiples para un mismo escenario.
- Conectar las opciones de respuesta de la vista con la lógica correspondiente del backend.

Permitir que el jugador identifique las señales que justifican su respuesta:

- Definir las posibles señales de fraude de cada escenario.
- Diseñar la interfaz para seleccionar las señales identificadas.
- Registrar las señales seleccionadas por el jugador.
- Comparar las señales seleccionadas con las correspondientes al escenario.

Evaluar automáticamente la respuesta del jugador:

- Implementar el procedimiento de evaluación.
- Comparar la clasificación seleccionada con la respuesta correcta.
- Evaluar las señales identificadas por el jugador.
- Determinar si la respuesta es correcta o incorrecta.
- Enviar el resultado al sistema de puntuación.

4. Aprendizaje y retroalimentación

Proporcionar retroalimentación que explique la respuesta, señale los indicadores de fraude y presente recomendaciones preventivas:

- Crear explicaciones para los escenarios.
- Identificar los indicadores de fraude correspondientes.
- Crear recomendaciones preventivas.
- Diseñar la pantalla de retroalimentación.
- Mostrar si la respuesta fue correcta o incorrecta.
- Mostrar los indicadores que el jugador debía reconocer.
- Permitir continuar al siguiente escenario.

5. Puntuación y progreso

Administrar los puntos e intentos del jugador según su desempeño:

- Definir las reglas para asignar puntos dependiendo del intento del usuario.
- Definir la cantidad de intentos disponibles.
- Actualizar el puntaje después de cada escenario.
- Actualizar los intentos según el desempeño.
- Mostrar el puntaje e intentos actuales.

Guardar automáticamente el progreso, puntuación, niveles completados y logros del usuario:

- Definir los datos que deben almacenarse.
- Implementar el almacenamiento del progreso.
- Guardar la puntuación.
- Guardar los niveles completados.
- Guardar los logros obtenidos.
- Recuperar los datos al iniciar sesión nuevamente.

Desbloquear nuevos niveles y logros cuando se cumplan las condiciones establecidas:

- Definir las condiciones para desbloquear niveles.
- Definir las condiciones para obtener logros.
- Verificar las condiciones después de cada partida o nivel.
- Actualizar los niveles disponibles.
- Registrar los logros obtenidos.
- Notificar al jugador cuando desbloquee un nivel o logro.
- Realizar pruebas de desbloqueo.

Mostrar un resumen del desempeño del jugador, incluyendo progreso, puntuación, aciertos, errores y logros:

- Registrar los aciertos y errores del jugador.
- Calcular el progreso alcanzado.
- Recuperar la puntuación y logros obtenidos.
- Diseñar la pantalla de resumen.
- Mostrar progreso, puntuación, aciertos, errores y logros.
- Implementar la opción para regresar al menú o continuar jugando.

6. Administración

Permitir al administrador acceder a las funciones administrativas:

- Identificar al usuario mediante su rol.
- Verificar que el usuario tenga permisos de administrador.
- Diseñar la vista para las funciones administrativas.
- Conectar la vista administrativa con las funciones correspondientes del backend.

Permitir al administrador gestionar los niveles del juego:

- Consultar los niveles existentes.
- Crear nuevos niveles.
- Modificar la información de un nivel.
- Desactivar un nivel.
- Guardar los cambios realizados.

Permitir al administrador gestionar los escenarios del juego:

- Consultar los escenarios disponibles.
- Crear nuevos escenarios.
- Modificar escenarios existentes.
- Desactivar escenarios.
- Asociar los escenarios con los niveles correspondientes.

- Guardar los cambios realizados.

7. Persistencia de datos

Almacenar y recuperar la información necesaria para el funcionamiento del sistema:

- Implementar la conexión entre el backend y la base de datos.
- Configurar el servidor necesario para la comunicación entre la aplicación y la base de datos.
- Almacenar los datos de los usuarios.
- Recuperar los datos del usuario durante el inicio de sesión.
- Almacenar la información de los niveles.
- Almacenar la información de los escenarios.
- Guardar el progreso, puntuación, niveles completados y logros del jugador.
- Recuperar los datos cuando sean necesarios.
- Actualizar la información almacenada cuando se realicen modificaciones.

Priorización de los requisitos

Prioridad 1:

Requisitos esenciales para el funcionamiento principal del juego.

- Iniciar una partida nueva.
- Presentar escenarios simulados de posibles estafas.
- Clasificar cada escenario como seguro o fraudulento.
- Identificar las señales que justifican la respuesta.
- Evaluar automáticamente la respuesta del jugador.
- Proporcionar retroalimentación sobre la respuesta y recomendaciones preventivas.

Prioridad 2:

Requisitos importantes que complementan el funcionamiento del juego.

- Permitir el registro y autenticación de usuarios.
- Mostrar las instrucciones y reglas del juego.
- Permitir seleccionar los niveles desbloqueados.
- Administrar los puntos e intentos del jugador.
- Continuar una partida guardada.
- Guardar el progreso, puntuación, niveles completados y logros.
- Mostrar un resumen del desempeño del jugador.
- Consultar y actualizar la información del usuario.
- Actualizar la contraseña del usuario.
- Conectar las vistas principales con las funciones correspondientes del backend.
- Almacenar y recuperar la información mediante la base de datos.

Prioridad 3:

Requisitos adicionales que mejoran, personalizan y permiten administrar la experiencia.

- Permitir al usuario seleccionar su perfil.
- Desbloquear nuevos niveles y logros.
- Permitir el acceso a las funciones administrativas según el rol del usuario.
- Gestionar los niveles desde las funciones de administrador.
- Gestionar los escenarios desde las funciones de administrador.
- Conectar las vistas administrativas con las funciones correspondientes del backend.